COMP282 – Project: Gaming Leaderboard

Languages allowed: Java **or** Python only.
Critical rule: You **may not** use any built-in data structure or algorithm libraries to implement the core storage/algorithms (no Java `java.util.*` collections; no Python `list`, `dict`, `set`, `tuple`, `heapq`, etc.).

- You **may** use: strings, primitive types, I/O, exceptions, and basic utilities (e.g., Java/Python random).
- For the **core storage**, implement from scratch:
  - A **Linked List** to store per-player score history (most-recent first).
  - A simple dynamic array (**Vector**) to hold snapshots for sorting.
  - Your own **Heap Sort** (build-heap + down-heap) operating on your Vector copy.

Goal

Build a command-line **Gaming Leaderboard** that ingests player scores over time, keeps a history per player, and can produce ranked leaderboards on demand using your **heap sort** (not the language's sort). The system must not mutate the underlying histories when creating sorted outputs.

Input & Command-Line Interface (CLI)

- **Invocation:** Your program must accept a single argument: the path to a plain-text commands file.
  - **Java:** `java Main commands.txt`
  - **Python:** `python3 main.py commands.txt`
- **Behavior when missing/invalid:** If the file path is missing or the file cannot be opened, print a one-line usage message and exit with a **non-zero** status code. Example:
  `USAGE: <program> <commands_file>`
- **Parsing rules:**
  - Each **non-empty** line in the file is a command to execute.
  - Lines beginning with `#` are **comments** and must be **ignored**.

  - Commands are **case-insensitive**; arguments are **case-sensitive** unless the spec says otherwise.
  - Arguments are **space-separated**; text wrapped in **double quotes** is treated as a single token (quotes not included in the value).
  - Extra whitespace at the beginning/end of lines should be ignored.
- **I/O requirements:**
  - All outputs must be printed to **stdout** exactly as specified by this spec.
  - Any invalid command or malformed arguments should print a helpful message prefixed with `ERROR:` and continue processing subsequent lines (unless the error is for `QUIT` handling).
  - The command `QUIT` should **terminate** processing immediately; any lines after `QUIT` are ignored.
- **No interactive mode:** Do **not** prompt for input; the entire run is driven solely by the provided file.

Functional Requirements

Commands are case-insensitive; arguments are space-separated. Player names use underscores in place of spaces or are wrapped in quotes.

1. **ADD_PLAYER name**
   Create a new player with an empty history and `best = -∞` (or `None` sentinel). If the player already exists, print `DUPLICATE`.
2. **ADD_SCORE name score**
   Append a score to the player's **Linked List** history (insert at head). Update the player's `best` if this score is higher. If player not found → `NOT FOUND`.
3. **CURRENT name**
   Print the player's most recent score (head of the list). If none → `NONE`. If player not found → `NOT FOUND`.
4. **BEST name**
   Print the player's best score so far (maintained incrementally). If player not found → `NOT FOUND`; if no scores → `NONE`.
5. **HISTORY name k**
   Print up to the last `k` scores from the player's linked list (most recent first). If player not found → `NOT FOUND`.
6. **TOP_K k**
   Produce a **snapshot leaderboard** of the **top k players by `best` score (descending)** using

your Heap Sort on a **Vector** copy of all players who have at least one score. Do not mutate player nodes.
    - **Tie-breaker: lexicographic by name.**
7. **PRINT_ALL**
   Print **all players** sorted by `best` descending using your heap sort snapshot (same tie-breaker). Players with no scores appear at the end with `best = NONE` (or skip them—pick one policy and document it).
8. **REMOVE_PLAYER name**
   Remove the player and their entire history. If not found → `NOT FOUND`.
9. **LEN**
   Print the number of players currently registered.
10. **CLEAR**
    Remove **all players** and their histories.
11. **QUIT**
    Exit cleanly.

Output Format

- **CURRENT/BEST**:
  `-> name | current=S | best=B`
- **HISTORY name k**: one per line, most recent first:
  `-> S`
- **TOP_K / PRINT_ALL**:
  `-> rank. name | best=B`

Use consistent formatting; handle empty cases with `EMPTY`.

Data Structure Constraints

- Implement your own **Player** record with:
  `name : String`, `best : int or sentinel`, `head : ScoreNode`, and (optional) `nextPlayer` if you maintain players in a linked list.
- **ScoreNode** (linked list): `value : int`, `next : ScoreNode`. Insert at head for O(1) append.
- Maintain players in **your own container**. You may choose either:

    - A **linked list** of players (simplest; O(n) lookup by name), **or**
    - A secondary **Vector** kept **sorted by name** for O(log n) binary search lookups (recommended if you want faster queries).
- Implement a **Vector** type with manual resize (e.g., 2× growth) to build snapshot arrays for heap sort.
- Implement **Heap Sort** yourself: build-heap in O(n), then repeatedly down-heap to produce descending order by `(best, -name)` with the required tie-breaker.
- Do **not** rely on language built-ins for dynamic arrays or sorting.

Example Session

```
ADD_PLAYER Alice
ADD_PLAYER Bob
ADD_PLAYER Carol

ADD_SCORE Alice 120
ADD_SCORE Bob 95
ADD_SCORE Alice 180
ADD_SCORE Carol 180
ADD_SCORE Bob 210

CURRENT Alice
-> Alice | current=180 | best=180

BEST Bob
-> Bob | best=210

HISTORY Alice 3
-> 180
-> 120

TOP_K 2
-> 1. Bob | best=210
-> 2. Alice | best=180   # tie vs Carol broken by name

PRINT_ALL
-> 1. Bob | best=210
-> 2. Alice | best=180
-> 3. Carol | best=180

LEN
3

REMOVE_PLAYER Carol
PRINT_ALL
-> 1. Bob | best=210
-> 2. Alice | best=180

CLEAR
LEN
```

```
0
QUIT
```

Performance Expectations

- **ADD_SCORE**: O(1) to push at head; O(1) to update `best`.
- **CURRENT/BEST/HISTORY**: O(1) to O(k) traversal of a single player's list.
- **TOP_K / PRINT_ALL**: O(p log p) where `p` is the number of players with at least one score (heap sort over snapshot).
- **ADD_PLAYER/REMOVE_PLAYER/lookup**: O(n) with linked-list registry, or O(log n) with a name-sorted Vector.

Design Sketch

Java (suggested)

```
class ScoreNode {
  int value;
  ScoreNode next;
}

class Player {
  String name;
  Integer best; // null if none
  ScoreNode head; // most recent first
  Player next;   // if you maintain a linked-list registry
}

class Vector<T> {
  Object[] data;
  int size, capacity;
  void pushBack(T x) { ... }
  T get(int i) { ... }
  void set(int i, T x) { ... }
  void swap(int i, int j) { ... }
  int length() { return size; }
}

class Leaderboard {
  Player head; // registry; or replace with a name-sorted Vector<Player>
  int count;

  Player find(String name) { /* O(n) scan or binary search in Vector */ }

  void heapSort(Vector<Player> v) {
    // build max-heap by (best desc, name asc), then extract
  }

  // command handlers: addPlayer, addScore, current, best, history, topK, printAll, rem
}
```

Python (suggested)

```python
class ScoreNode:
    __slots__ = ("value","next")
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

class Player:
    __slots__ = ("name","best","head","next")
    def __init__(self, name):
        self.name = name
        self.best = None
        self.head = None
        self.next = None  # optional registry list

class Vector:
    def __init__(self, cap=4):
        self._data = [None]*cap  # fixed buffer; grow manually
        self._size = 0
    # push_back, get, set, swap, _grow

def better(a: Player, b: Player) -> bool:
    # return True if a should come before b
    # treat None best as -infinity; then compare name ascending as tiebreaker
    ...

def heap_sort(players_vector: Vector):
    # build heap using better(), then extract to descending order
    ...

class Leaderboard:
    def __init__(self):
        self.head = None  # or maintain a Vector for name-sorted registry
        self.count = 0
    # add_player, add_score, current, best, history, top_k, print_all, remove_player, c
```

Edge Cases & Rules

- Scores must be integers (or define your policy for floats). Non-numeric → `INVALID` .
- Adding a score for an unknown player → `NOT FOUND` .
- `TOP_K` larger than number of available players → return all.
- Players with no scores: either exclude from rankings or include with `best = NONE` at the end (choose and document).
- All outputs should be deterministic under ties (use the specified tie-breaker).

Deliverables

1. **Source code** (Java or Python).
2. **README** (≤1 page) with:
    - Linked-list/Vector/heap-sort design and invariants.
    - How to compile/run.
    - Commands implemented & formatting details.
3. **Test transcript**: a short session covering commands and edge cases.

Academic Integrity & Restrictions

"Cheating or plagiarism in connection with an academic program at a campus is listed in Section 41301, Title V, California Code of Regulations, as an offense for which a student may be expelled, suspended, or given a less severe disciplinary sanction." -University Catalog
In this course, all submitted code and written work must be your own. Sharing, copying, or submitting work created by another individual (including material from the internet or AI tools without authorization) will be treated as academic dishonesty.

For full details, refer to the http://www.csun.edu/catalog

- Implementations must be **original**.
- **Disallowed**: Java Collections (`List` , `ArrayList` , `LinkedList` , `Map` , `Set` , `Deque`), Python containers (`list` , `dict` , `set` , `tuple` , `deque` , `heapq`) as core storage/algorithms.
- **Allowed**: Strings, primitives, I/O, exceptions, and your own Linked List, Vector, and Heap Sort implementations.